

Course Code	Course Title	L	R	P	Cr
	AI-Assisted Problem Solving Using Python Programming				
Program Core					

Day 3 Session: Loops, Strings & Text Processing

Day 3

- while & for Loops
- Loop Control Statements
- String Operations
- String Formatting

Topics

- while Loops
- for Loops
- Loop Control Statements
- String Operations
- String Formatting

Question : Can a computer automatically process thousands of words without writing the same code repeatedly?

Examples

- Password checker
- Email validator
- Chatbot
- Text analyzer
- Search engine

Without loops:

```
print("Welcome to SRU")
print(" Welcome to SRU ")
print(" Welcome to SRU ")
print(" Welcome to SRU ")
print(" Welcome to SRU ")
```

Benefits:

- Less code
- Easy maintenance
- Faster execution
- Automation

With loops:

```
for i in range(5):
    print("Welcome to SRU")
```

or

```
i=1
while(i<=5):
    print ("Welcome to SRU")
```

Python supports

while Loop: Condition-controlled

while condition:
 statements

for Loop: Sequence-controlled

for variable in sequence/range:
 statements

while Loop Flowchart

Syntax:

initialization

while condition:

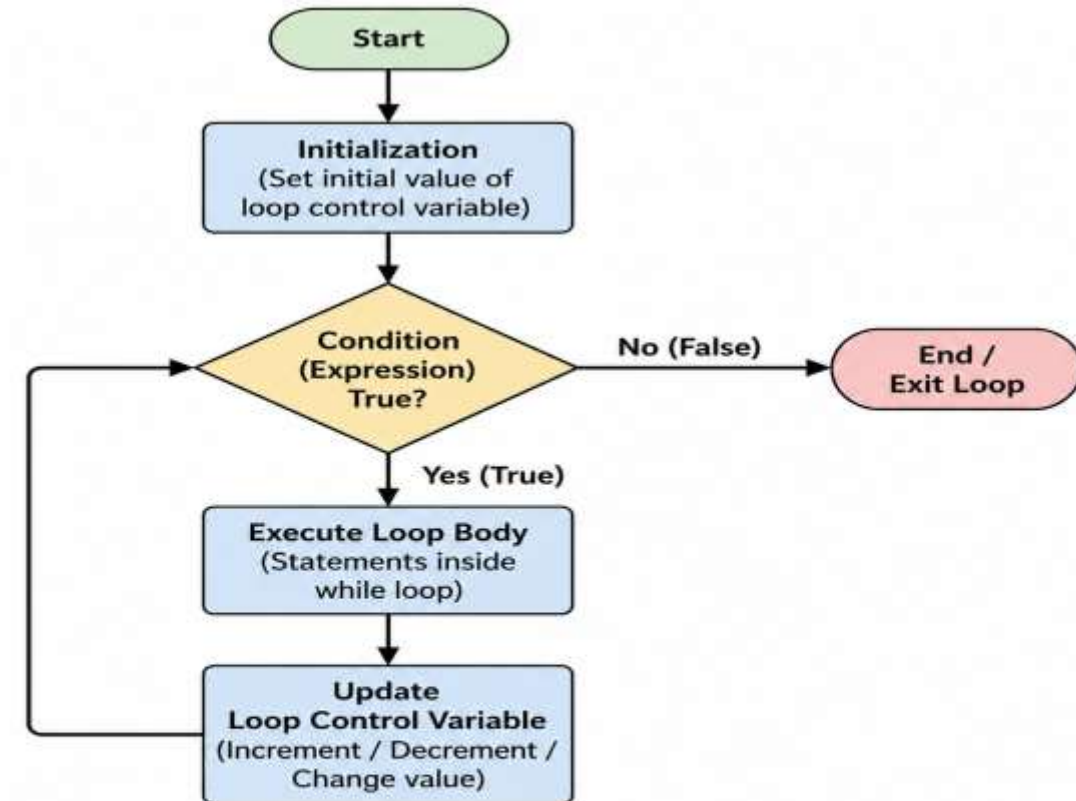
statement1

statement2

...

statementN

increment / decrement



While Condition is True:

Execute the loop body and update the loop control variable.
When Condition becomes False, the loop terminates.

Example of while Loop:

Basic Example (Counting)

This loop prints numbers from 1 to 5.

```
count = 1
while count <= 5:
    print(count)
    count += 1 # Increments the count, eventually making the condition False
print("Loop finished")
```

Output:

```
1
2
3
4
5
Loop finished
```

Infinite Loop

```
while True:  
    print("Python")
```

Question

Why doesn't it stop?

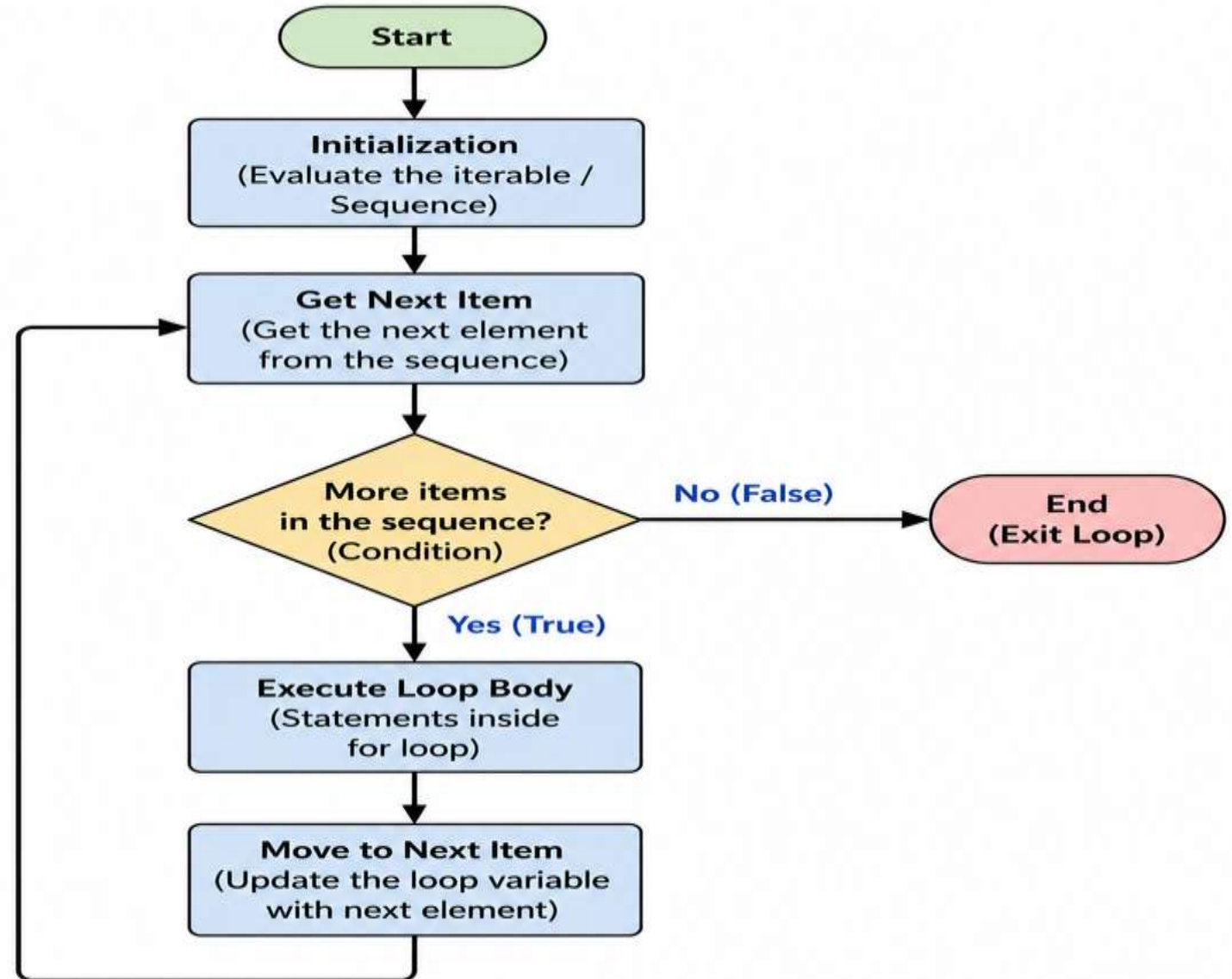
Ctrl+C interruption.

Syntax

for variable in iterable:
statements

Iterables:

- range()
- String
- List
- Tuple
- Dictionary



range() Function: The range() is a built-in function in Python.

Syntax: range(start, stop, step)

Examples

range(5)

Output

0 1 2 3 4

```
list(range(10))
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

#start value is given as 2

```
list(range(2, 10)) #[2, 3, 4, 5, 6, 7, 8, 9]
```

```
[2, 3, 4, 5, 6, 7, 8, 9]
```

range(2,10)

Output

2 3 4 5 6 7 8 9

#step value is 5 and start value is 0

```
list(range(0, 30, 5)) #[0, 5, 10, 15, 20, 25]
```

```
[0, 5, 10, 15, 20, 25]
```

range(2,20,2)

Output

2 4 6 8 ...

#step value is -1. Hence, decreasing

#sequence is generated

```
list(range(0, -9, -1)) #[0, -1, -2, -3, -4, -5, -6, -7, -8]
```

```
[0, -1, -2, -3, -4, -5, -6, -7, -8]
```

for Loop Example:

```
for i in range(1,6):  
    print(i)
```

Output

1
2
3
4
5

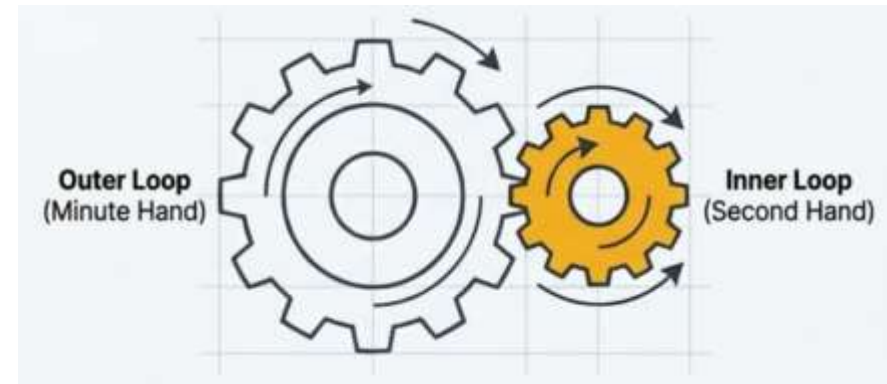
What is a Nested Loop ?

A nested loop is a loop inside another loop.

- The outer loop controls the number of iterations
- The inner loop completes all its iterations for each iteration of the outer loop

The Clock Analogy

- Think of the outer loop as the minute hand. It moves one step at a time.
- Think of the inner loop as the second hand. It must complete a full 60-tick rotation before the minute hand can advance to the next minute.



Nested Loops:

```
for i in range(3):  
    for j in range(3):  
        print(i, j)
```

Applications

- Tables
- Patterns
- Matrix
- Games

Pattern Printing:

```
for i in range(5):  
    print("*"*i)
```

```
*  
* *  
* * *  
* * * *
```

```
for i in range(5):  
    for j in range(i):  
        print("*", end="")  
    print()
```

```
*  
* *  
* * *  
* * * *
```

Nested Loops: How They Execute

Code Example

```
# The outer loop iterates from 0 to 2
for i in range(3):
    # The inner loop iterates from 0 to 1
    for j in range(2):
        print(f"i = {i}, j = {j}")
```

Execution Trace

- **Outer loop starts: $i = 0$**
 - **Inner loop runs completely:**
 - $j = 0$. Output: $i = 0, j = 0$
 - $j = 1$. Output: $i = 0, j = 1$
- **Outer loop continues: $i = 1$**
 - **Inner loop runs again:**
 - $j = 0$. Output: $i = 1, j = 0$
 - $j = 1$. Output: $i = 1, j = 1$
- **Outer loop continues: $i = 2$**
 - **Inner loop runs a final time:**
 - $j = 0$. Output: $i = 2, j = 0$
 - $j = 1$. Output: $i = 2, j = 1$

The inner loop runs 2 times for each of the 3 iterations of the outer loop, for a total of $3 * 2 = 6$ print statements.

Printing a Square Pattern

```
for i in range(3):
    for j in range(3):
        # end=" " prevents a new line
        print("*", end=" ")
    # This print() creates a new line
    # after each row
    print()
```

```
* * *
* * *
* * *
```

Generating a Multiplication Table

```
for i in range(1, 6):
    for j in range(1, 6):
        # end="\t" adds a tab for spacing
        print(i * j, end="\t")
    print()
```

1	2	3	4	5
2	4	6	8	10
3	6	9	12	15
4	8	12	16	20
5	10	15	20	25

Print Coordinate Pairs

```
for x in range(1, 4):  
    for y in range(1, 4):  
        if x != y:  
            print(f"({x}, {y})")
```

```
(1, 2)  
(1, 3)  
(2, 1)  
(2, 3)  
(3, 1)  
(3, 2)
```

while inside while

```
▶ i = 1
  while i <= 3:
    j = 1
    while j <= 2:
      print(i, j)
      j += 1
    i += 1
```

```
... 1 1
     1 2
     2 1
     2 2
     3 1
     3 2
```

Three statements

- break
- continue
- pass

break Statement : Stops immediately.

```
for i in range(10):  
    if i == 5:  
        break  
    print(i)
```

Output

0
1
2
3
4

continue Statement: Skips one iteration.

```
for i in range(6):  
    if i == 3:  
        continue  
    print(i)
```

Output

0
1
2
4
5

pass Statement: Placeholder for future code.

```
for i in range(5):  
    if i==2:  
        pass  
    print(i)
```

'break' only exits inner loop

```

▶ for i in range(3):
    for j in range(3):
        if j == 1:
            break
        print(i, j)

```

```

... 0 0
     1 0
     2 0

```

'break' exits only the nearest loop

'continue' in inner loop

```

▶ for i in range(3):
    for j in range(3):
        if j == 1:
            continue
        print(i, j)

```

```

0 0
0 2
1 0
1 2
2 0
2 2

```

What is for-else?

- In Python, a for loop can have an else block.
- The else block executes only if the loop finishes normally (without break)

```
for item in sequence:  
    if condition:  
        break  
else:  
    statement
```

Searching an Element

```
▶ numbers = [2, 4, 6, 8]
key = 5

for num in numbers:
    if num == key:
        print("Element found")
        break
else:
    print("Element not found")
```

Element not found

Prime Numbers in a Range

```
▶ for num in range(2, 10):  
    for i in range(2, num):  
        if num % i == 0:  
            break  
    else:  
        print(num, "is prime")
```

```
2 is prime  
3 is prime  
5 is prime  
7 is prime
```

What is while-else ?

- Just like for, while can also have an else.
- else executes only if the loop ends naturally

```
while condition:  
    if condition2:  
        break  
else:  
    statement
```

Countdown without break

```
i = 3
while i > 0:
    print(i)
    i -= 1
else:
    print("Countdown completed")
```

```
3
2
1
Countdown completed
```

- Nested loops allow one loop to run completely inside another, increasing the total number of iterations.
- The inner loop always completes all its iterations for each iteration of the outer loop.
- In Python, loops (for and while) can have an else block.
- The else block of a loop executes only when the loop terminates normally, not when it exits using break.
- for–else is commonly used in searching and prime number problems.
- while–else is useful in retry-based logic such as authentication and input validation.
- break affects only the nearest enclosing loop in nested structures.

Strings in python are surrounded by either single quotation marks, or double quotation marks. 'hello' is the same as "hello".

You can display a string literal with the print() function:

Example:

```
#You can use double or single quotes:  
  
print("Hello")  
print('Hello')
```

```
Hello  
Hello
```

Assign String to a Variable:

Assigning a string to a variable is done with the variable name followed by an equal sign and the string:

Example:

```
a = "Hello"  
print(a)
```

```
Hello
```

Single Line String Creation

```
[1]
✓ 0s
▶ # Single-quoted and double-quoted strings
name = 'Hello'
greeting = "Welcome to Python!"
print(name)      # Output: Alice
print(greeting)  # Output: Welcome to Python!
```

```
▼ ... Hello
    Welcome to Python!
```

Multi Line String Creation

```
▶ # Triple-quoted multi-line string
multiline1 = """This is
a multi-line
string with double quotes."""

multiline2 = '''This is
a multi-line
string with single quotes.'''

print(multiline1)

print(multiline2)
```

```
... This is
    a multi-line
    string with double quotes.
    This is
    a multi-line
    string with single quotes.
```

Creating a raw string

```
▶ # Raw string
raw_path = r'C:\newfolder\data'
print(raw_path)    # Output: C:\newfolder\data
```

```
... C:\newfolder\data
```

Checking datatype of string

```
▶ string1 = "Hello, Python"
string2 = ""
print(type(string1))
print(type(string2))
```

```
... <class 'str'>
    <class 'str'>
```

Creating a raw string

```

▶ # Raw string
raw_path = r'C:\newfolder\data'
print(raw_path)    # Output: C:\newfolder\data

... C:\newfolder\data

```

The `str()` Constructor

```

▶ # Creating a string from an integer
year = 2025
year_str = str(2025)
print(year)
print(year_str)
# year_str is now "2025"
print(type(year))
print(type(year_str))

```

```

2025
2025
<class 'int'>
<class 'str'>

```

Checking datatype of string

```

▶ string1 = "Hello, Python"
string2 = ""
print(type(string1))
print(type(string2))

```

```

... <class 'str'>
    <class 'str'>

```


Accessing Characters in Strings

- Strings in Python are sequences, so individual characters can be accessed using indexing.
- Python follows 0-based indexing, where the first character is at index 0.
- Negative indexing allows access from the end of the string (-1 for last character, -2 for second-to-last).
- Accessing an index outside the valid range raises an `IndexError`.
- The built-in `len()` function returns the length of a string, and the last valid index is `len(s) - 1`.

Accessing Characters by Index

Key Concepts

- Positive Indexing: Starts from 0 for the first character. `s[0]` is the first character.
- Negative Indexing: Starts from -1 for the last character. `s[-1]` is the last character.
- `len()` function: Use `len(s)` to get the total number of characters.
- `IndexError`: Accessing an index outside the valid range raises an `IndexError`.

0	1	2	3	4	5
P	y	t	h	o	n
-6	-5	-4	-3	-2	-1

Positive, Negative Indexing & Indexing Error

```
# Using positive and negative indices to access characters
s = "Python"
print(s[0]) # Output: 'P' (First character)
print(s[2]) # Output: 't' (Third character)
print(s[-1]) # Output: 'n' (Last character)
print(s[7]) # Output: IndexError

... P
    t
    n

-----
IndexError                                Traceback (most recent call last)
/tmp/ipython-input-169961637.py in <cell line: 0>()
      4 print(s[2]) # Output: 't' (Third character)
      5 print(s[-1]) # Output: 'n' (Last character)
----> 6 print(s[7]) # Output:

IndexError: string index out of range
```

Using `len()` and Accessing the Last Character

```
# Finding the length and using it to get the
word = "hello"
length = len(word) # length is 5
last_char = word[length - 1]
print(last_char)
# Output: o

... 0
```

Strings are Arrays

- Like many other popular programming languages, strings in Python are arrays of bytes representing unicode characters.
- However, Python does not have a character data type, a single character is simply a string with a length of 1.
- Square brackets can be used to access elements of the string.

-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1	
S	R	U	N	I	V	E	R	S	I	T	Y	
0	1	2	3	4	5	6	7	8	9	10	11	

```
a = "SRUNIVERSITY"
print(a[4]) # prints 4 indexed element i.e, 5th element of the given string
print(a[-2]) # prints last second element
```

The Principle of Immutability

- Strings in Python are immutable, meaning their contents cannot be changed after creation.
- Assigning a new value to a specific index (for example, `s[0] = 'x'`) raises a `TypeError`.
- Any operation that appears to modify a string actually creates a new string.
- To change a string, a new string must be constructed using slicing or string methods.
- String immutability allows strings to be safely used as dictionary keys and helps prevent unintended bugs.

Demonstrating the Error

```
# Attempting to modify a string in place raises
msg = "Hello"
msg[0] = 'h' # This will fail
```

```
-----
TypeError                                 Traceback (most recent call last)
/tmp/ipython-input-2563037285.py in <cell line: 0>()
      1 # Attempting to modify a string in place raises
      2 msg = "Hello"
----> 3 msg[0] = 'h' # This will fail
```

```
TypeError: 'str' object does not support item assignment
```

The Correct "Modification" Pattern

```
▶ # To "change" a string, you must create a new
   title = "Mr Smith"
   # Create a new string by concatenating parts of
   new_title = "Dr " + title[3:]
   print(new_title) # Output: Dr Smith
   print(title) # Output: Mr Smith (The original
```

```
Dr Smith
Mr Smith
```

- ✓ **Concatenation**
- ✓ **Repetition**
- ✓ **Membership**
- ✓ **Slicing**
- ✓ **String Formatting**

Concatenation (`+`)

- Joins two or more strings together end-to-end.

Note: You can only concatenate strings with other strings. Use `str()` to convert other types first.



String Concatenation

```
# Joining multiple strings to form a complete sentence
first = "Data"
second = "Science"
full = first + " " + second
print(full) # Output: Data Science
```

Data Science

Concatenation of strings with integers

```
▶ name = "Alice"
age = 30

# This will raise a TypeError
print("Name: " + age)

# Correct way using str()
# print("Name: " + name + ", Age: " + str(age))

...
TypeError                                Traceback (most recent call last)
/tmp/ipython-input-3282866707.py in <cell line: 0>()
      3
      4 # This will raise a TypeError
----> 5 print("Name: " + age)
      6
      7 # Correct way using str()

TypeError: can only concatenate str (not "int") to str
```

```
▶ name = "Alice"
age = 30

# This will raise a TypeError
# print("Name: " + age)

# Correct way using str()
print("Name: " + name + ", Age: " + str(age))

... Name: Alice, Age: 30
```


Repetition (`\*`)

- Repeats a string a specified number of times.



Repetition

```
# Using the repetition operator for patterns  
laugh = "ha"  
print(laugh * 3) # Output: hahaha  
divider = "_" * 20  
print(divider) # Output: -----
```

hahaha

Membership Operators in Python Strings

Membership operators are used to **check whether a character or substring exists in a string**.

Python provides two membership operators:

in → Returns True if the substring is present.

not in → Returns True if the substring is not present.

Example 1: Using `in`

```
</> Python
text = "Python Programming"

print("Python" in text)
print("Program" in text)
print("Java" in text)
```

Output:

```
True
True
False
```

Example 2: Using `not in`

```
</> Python
language = "Python"

print("Java" not in language)
print("Py" not in language)
```

Output:

```
True
False
```

Example 3: Checking a Character

```
</> Python

word = "Computer"

print("C" in word)
print("m" in word)
print("z" in word)
```

Output:

```
True
True
False
```

Extracting Substrings with Slicing

- String slicing is used to extract a substring using the syntax `s[start:end:step]`.
- The start index is inclusive, while the end index is exclusive.
- If start is omitted, slicing begins from index 0.
- If end is omitted, slicing continues until `len(s)`.
- If step is omitted, it defaults to 1, selecting every character.
- Negative values for start or end count positions from the end of the string.
- A negative step reverses the slicing direction, commonly used to reverse a string (e.g., `s[::-1]`).

Basic and Default Slicing

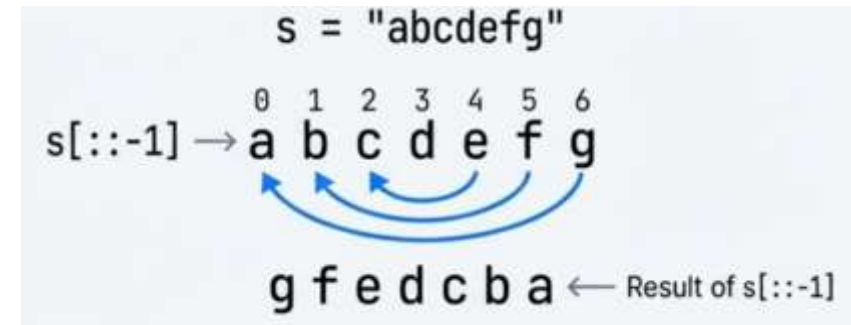
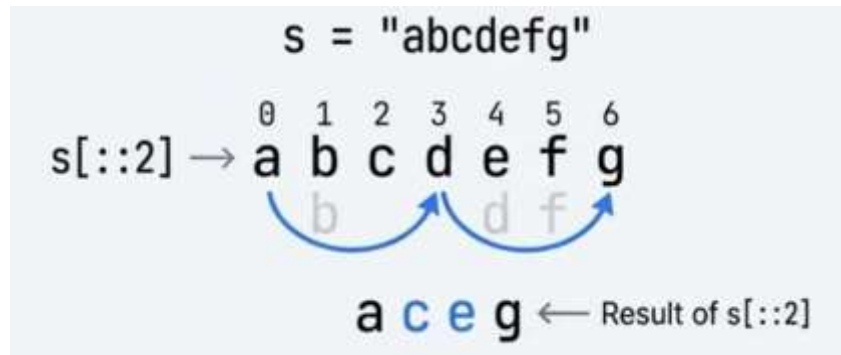
```
# Slicing to get specific parts of a string
text = "Hello, world!"
print(text[0:5]) # Output: 'Hello' (indices 0, 1, 2, 3, 4)
print(text[7:])
print(text[:5])
# Output: 'world!' (from index 7 to the end)
# Output: 'Hello' (from the beginning up to index 5)
```

```
... Hello
   world!
   Hello
```

Slicing with a Step

```
# Using a step to skip characters or reverse the string
s = "abcdefg"
print(s[::2]) # Output: 'aceg' (every second character)
print(s[::-1]) # Output: 'gfedcba' (the entire string in reverse)
```

```
aceg
gfedcba
```



Formatting Strings

- Python supports multiple string formatting techniques to embed values within strings.
- Old-style (%) formatting uses placeholders such as %s, %d, and %f and inserts values using the % operator.
- The str.format() method uses {} placeholders and allows positional or named arguments for flexible formatting.
- F-strings, introduced in Python 3.6, embed variables and expressions directly inside {} and support format specifiers.
- All formatting methods create new strings, and f-strings are generally the most concise and readable approach.

Old-Style '% ' Formatting

Uses % placeholders (%s, %d). Older, less flexible, but still seen in legacy code.

```
name = "Alice"
age = 30
print("Name: %s, Age: %d" % (name, age))
# Output: Name: Alice, Age: 30
```

Name: Alice, Age: 30

The .format() Method

Uses {} placeholders and is called on the string. More powerful than % formatting.

```
pi = 3.14159
print("Pi is approximately {:.2f}".format(pi))
# Output: Pi is approximately 3.14
```

Pi is approximately 3.14

F-Strings (Formatted String Literals)

The most modern, concise, and efficient method. Prefix the string with f and place expressions directly inside {}.

```
city = "Paris"
temp = 20
print(f"The temperature in {city} is {temp}°C.")
# Output: The temperature in Paris is 20°C.
```

... The temperature in Paris is 20°C.

```
name="Anil"

cgpa=8.76

print(f"{name} secured CGPA {cgpa}")
```

Common String Methods : upper() & lower()

- Python's string methods allow easy transformation.
- upper(): returns a new string with all characters converted to uppercase.
- lower(): returns a new string with all characters converted to lowercase.
- These methods are often used for case-insensitive comparisons or formatting. They do not change the original string (since strings are immutable)

```
word = "PyThOn"
print(word.upper())    # Output: 'PYTHON'
print(word.lower())    # Output: 'python'
```

```
PYTHON
python
```

```
text = "Hello World!"
upper_text = text.upper()
lower_text = text.lower()
print(upper_text)      # Output: 'HELLO WORLD!'
print(lower_text)      # Output: 'hello world!'
```

```
HELLO WORLD!
hello world!
```


Common String Methods: strip()

- **strip([chars]):** Returns a new string with leading and trailing whitespace (spaces, tabs, newlines) removed. If chars is provided, it removes those characters from both ends.

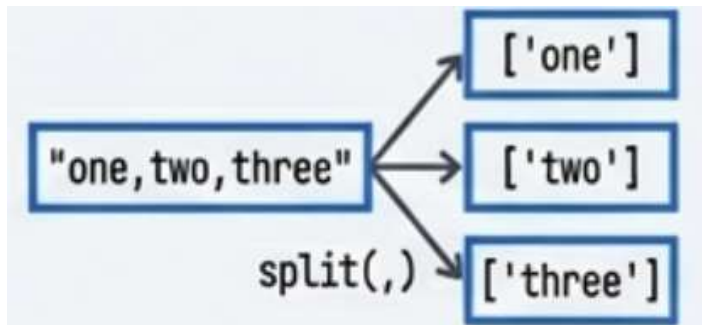


```
s = "  hello  "
print(s.strip())           # Output: 'hello'
print(">>>" + s.strip() + "<<<") # '>>>hello<<<'
path = "/home/user/"
print(path.strip("/"))     # Output: 'home/user'
```

```
hello
>>>hello<<<
home/user
```

Common String Methods: split()

- **split(sep=None):** Splits the string into a list of substrings using sep as the delimiter (default is any whitespace).

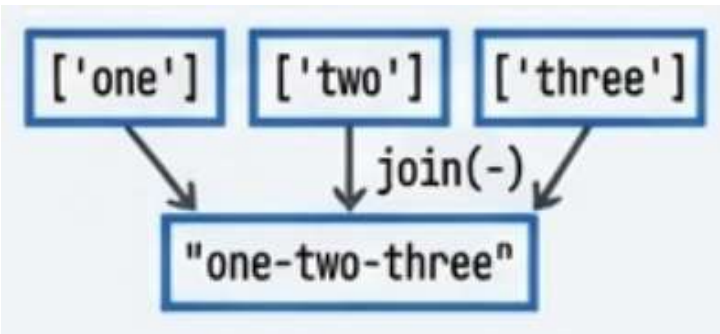


```
text = "apple,banana,cherry"
fruits = text.split(",")
print(fruits)                # Output: ['apple', 'banana', 'cherry']
sentence = "This is a test"
words = sentence.split()     # default splits on whitespace
print(words)                 # Output: ['This', 'is', 'a', 'test']
```

```
['apple', 'banana', 'cherry']
['This', 'is', 'a', 'test']
```

Common String Methods: join()

- **join(iterable):** Opposite of split(). It concatenates an iterable of strings (e.g. a list) into one string, with the original string as the separator.



▶

```

words = ["Python", "is", "awesome"]
sentence = " ".join(words)
print(sentence)           # Output: 'Python is awesome'
numbers = ["1", "2", "3"]
print(",".join(numbers))  # Output: '1,2,3'
  
```

```

... Python is awesome
   1,2,3
  
```

Common String Methods: find() & replace()

- **find(sub, [start], [end]):** Searches for the substring sub and returns the lowest index where it's found, or -1 if not found. (Unlike index(), it does not raise an error if not found.)
- **replace(old, new, [count]):** Returns a new string where all (or up to count) occurrences of substring old are replaced with new.



The diagram illustrates the replace() method. It shows the string "hello world" where "world" is highlighted in a blue box. An arrow points down to the string "hello Python", where "Python" is also highlighted in a blue box, indicating that "world" has been replaced by "Python".

```
text = "Hello, world"
print(text.find("world"))    # Output: 7 (index where 'world' starts)    7
print(text.find("x"))       # Output: -1 (not found)                    -1
print(text.find("o", 5))    # Output: 8 (find 'o' starting at index 5)  8

s = "Hello Hello"
print(s.replace("Hello", "Hi"))    # Output: 'Hi Hi'
print(s.replace("l", "*", 2))      # Output: 'He**o Hello' (replace first 2 'l's)
text = "spam eggs spam"
print(text.replace("spam", "ham")) # Output: 'ham eggs ham'
```

```
Hi Hi
He**o Hello
ham eggs ham
```

Multiple Methods - 1

```
sentence = "  Data Science is fun!  "
trimmed = sentence.strip()           # remove outer whitespace
print("Trimmed:", trimmed)           # 'Data Science is fun!'
words = trimmed.split()              # split into words
print("Words list:", words)           # e.g. ['Data', 'Science', 'is', 'fun!']
joined = "-".join(words)              # join with dashes
print("Joined with dashes:", joined)  # 'Data-Science-is-fun!'
index = joined.find("fun")
print("Index of 'fun':", index)        # e.g. 17
replaced = joined.replace("fun", "awesome")
print("Replaced string:", replaced)    # 'Data-Science-is-awesome!'
```

```
Trimmed: Data Science is fun!
Words list: ['Data', 'Science', 'is', 'fun!']
Joined with dashes: Data-Science-is-fun!
Index of 'fun': 16
Replaced string: Data-Science-is-awesome!
```


Multiple Methods - 2

```
greet = "Hi"  
name = "Bob"  
message = f"{greet}, {name}!"  
print(message) # 'Hi, Bob!'  
print("Length:", len(message)) # e.g. 7  
print("Uppercase:", message.upper()) # 'HI, BOB!'
```

Hi, Bob!
Length: 8
Uppercase: HI, BOB!

Your Reference Toolkit: The Complete String Method List

All string methods return a new value and do not modify the original string. Use this guide for quick reference.

Case Conversion

capitalize()
casefold()
lower()
upper()
title()
swapcase()

Searching & Checking (`is\*` methods)

find()	isalnum()
rfind()	isalpha()
index()	isdigit()
rindex()	islower()
count()	isupper()
startswith()	isspace()
endswith()	istitle()

Splitting & Joining

split()
rsplit()
splitlines()
partition()
rpartition()
join()

Trimming & Padding

strip()
lstrip()
rstrip()
center()
ljust()
rjust()
zfill()

Replacing & Translating

replace()
translate()
maketrans()

Formatting

format()
format_map()
expandtabs()

For a complete list and advanced options, always refer to the official Python documentation.

Method	Description
<code>capitalize()</code>	Converts the first character to upper case
<code>casefold()</code>	Converts string into lower case
<code>center()</code>	Returns a centered string
<code>count()</code>	Returns the number of times a specified value occurs in a string
<code>encode()</code>	Returns an encoded version of the string
<code>endswith()</code>	Returns true if the string ends with the specified value
<code>expandtabs()</code>	Sets the tab size of the string
<code>find()</code>	Searches the string for a specified value and returns the position of where it was found
<code>format()</code>	Formats specified values in a string
<code>format_map()</code>	Formats specified values in a string

<code>index()</code>	Searches the string for a specified value and returns the position of where it was found
<code>isalnum()</code>	Returns True if all characters in the string are alphanumeric
<code>isalpha()</code>	Returns True if all characters in the string are in the alphabet
<code>isdecimal()</code>	Returns True if all characters in the string are decimals
<code>isdigit()</code>	Returns True if all characters in the string are digits
<code>isidentifier()</code>	Returns True if the string is an identifier
<code>islower()</code>	Returns True if all characters in the string are lower case
<code>isnumeric()</code>	Returns True if all characters in the string are numeric
<code>isprintable()</code>	Returns True if all characters in the string are printable
<code>isspace()</code>	Returns True if all characters in the string are whitespaces
<code>istitle()</code>	Returns True if the string follows the rules of a title
<code>isupper()</code>	Returns True if all characters in the string are upper case
<code>join()</code>	Joins the elements of an iterable to the end of the string
<code>ljust()</code>	Returns a left justified version of the string
<code>lower()</code>	Converts a string into lower case
<code>lstrip()</code>	Returns a left trim version of the string
<code>maketrans()</code>	Returns a translation table to be used in translations

<code>partition()</code>	Returns a tuple where the string is parted into three parts
<code>replace()</code>	Returns a string where a specified value is replaced with a specified value
<code>rfind()</code>	Searches the string for a specified value and returns the last position of where it was found
<code>rindex()</code>	Searches the string for a specified value and returns the last position of where it was found
<code>rjust()</code>	Returns a right justified version of the string
<code>rpartition()</code>	Returns a tuple where the string is parted into three parts
<code>rsplit()</code>	Splits the string at the specified separator, and returns a list
<code>rstrip()</code>	Returns a right trim version of the string
<code>split()</code>	Splits the string at the specified separator, and returns a list
<code>splitlines()</code>	Splits the string at line breaks and returns a list
<code>startswith()</code>	Returns true if the string starts with the specified value
<code>strip()</code>	Returns a trimmed version of the string
<code>swapcase()</code>	Swaps cases, lower case becomes upper case and vice versa
<code>title()</code>	Converts the first character of each word to upper case
<code>translate()</code>	Returns a translated string
<code>upper()</code>	Converts a string into upper case
<code>zfill()</code>	Fills the string with a specified number of 0 values at the beginning

```
s = "hello world"

# capitalize(): Capitalizes first character, rest become lowercase
print(s.capitalize())          # Hello world

# casefold(): Aggressive lowercase conversion (useful for comparisons)
print("Straße".casefold())      # strasse

# center(): Centers string within given width using fill character
print(s.center(20, "*"))        # ****hello world****

# count(): Counts occurrences of a substring
print(s.count("l"))             # 3

# encode(): Converts string to bytes using encoding
print(s.encode("utf-8"))        # b'hello world'

# endswith(): Checks if string ends with given suffix
print(s.endswith("world"))      # True

# expandtabs(): Replaces tabs with spaces
print("A\tB".expandtabs(4))     # A    B
```

```
# find(): Returns lowest index of substring or -1
print(s.find("world"))          # 6

# format(): Formats string using {} placeholders
print("My name is {}".format("Alice")) # My name is Alice

# format_map(): Formats string using dictionary
data = {"name": "Alice", "age": 30}
print("{name} is {age}".format_map(data)) # Alice is 30

# index(): Like find(), but raises error if not found
print(s.index("hello"))        # 0

# isalnum(): True if all characters are alphanumeric
print("Python3".isalnum())     # True

# isalpha(): True if all characters are alphabets
print("Python".isalpha())      # True

# isascii(): True if all characters are ASCII
print(s.isascii())            # True

# isdecimal(): True if all characters are decimal digits
print("123".isdecimal())       # True

# isdigit(): True if all characters are digits
print("123".isdigit())         # True
```



```
# rjust(): Right-justifies string within width
print("Hi".rjust(5, "*"))      # ***Hi

# rpartition(): Splits at last occurrence of separator
print("a-b-c".rpartition("-")) # ('a-b', '-', 'c')

# rsplit(): Splits string from right
print("a,b,c".rsplit(",", 1))  # ['a,b', 'c']

# rstrip(): Removes trailing whitespace or characters
print("hello!!!".rstrip("!"))  # hello

# split(): Splits string into list
print("Python is fun".split()) # ['Python', 'is', 'fun']

# splitlines(): Splits string at line breaks
print("A\nB\nC".splitlines())  # ['A', 'B', 'C']

# startswith(): Checks prefix
print(s.startswith("hello"))  # True

# strip(): Removes leading and trailing characters
print("***hello***".strip("*")) # hello

# swapcase(): Swaps uppercase to lowercase and vice versa
print("Hello PYTHON".swapcase()) # hELLO python

# title(): Converts string to title case
print("hello world".title())   # Hello World
```

```
# lower(): Converts string to lowercase
print("HELLO".lower())      # hello

# lstrip(): Removes leading whitespace or characters
print("  hello".lstrip())   # hello

# maketrans(): Creates translation table
table = str.maketrans("aeiou", "12345")

# translate(): Replaces characters using translation table
print("education".translate(table)) # 2d5c1t45n

# partition(): Splits at first occurrence of separator
print("a-b-c".partition("-")) # ('a', '-', 'b-c')

# removeprefix(): Removes prefix if present (Python 3.9+)
print("unhappy".removeprefix("un")) # happy

# removesuffix(): Removes suffix if present (Python 3.9+)
print("filename.txt".removesuffix(".txt")) # filename

# replace(): Replaces substring with another
print(s.replace("world", "Python")) # hello Python

# rfind(): Finds last occurrence index
print("banana".rfind("a"))      # 5

# rindex(): Like rfind(), but raises error if not found
print("banana".rindex("a"))     # 5
```

```
# upper(): Converts string to uppercase
print("hello".upper())        # HELLO

# zfill(): Pads string with zeros on left
print("123".zfill(6))        # 000123
```

Python Escape Characters

1. \ ' → Single Quote

Used to include a single quote inside a single-quoted string

```
text = 'It\'s a sunny day'
```

```
print(text)
```

Output: It's a sunny day

Explanation: \ ' allows the single quote in "It's" without ending the string

2. \\ → Backslash

Used to represent a single backslash character

```
path = "C:\\Users\\Sudhir\\Documents"
```

```
print(path)
```

Output: C:\Users\Sudhir\Documents

Explanation: Each \\ is interpreted as one literal backslash

3. \n → New Line

Inserts a line break

```
text = "Hello\nWorld"
```

```
print(text)
```

Output:

Hello

World

Explanation: \n moves the cursor to the next line

Escape Characters

Other escape characters used in Python:

Code	Result
\'	Single Quote
\\	Backslash
\n	New Line
\r	Carriage Return
\t	Tab
\b	Backspace
\f	Form Feed
\ooo	Octal value
\xhh	Hex value


```
# 4. \r → Carriage Return
# Moves cursor to the beginning of the line
text = "Hello\rHi"
print(text)
# Output (typical console): Hillo
# Explanation:
# "Hello" is printed first
# \r moves cursor to start of line
# "Hi" overwrites the first two characters → Hillo

# 5. \t → Tab
# Inserts a horizontal tab space
text = "Name:\tSudhir"
print(text)
# Output: Name:   Sudhir
# Explanation: \t inserts a tab for spacing/alignment

# 6. \b → Backspace
# Removes the previous character
text = "Hello\b"
print(text)
# Output: Hello
# Explanation: \b removes the character just before it
```



```
# 7. \f → Form Feed
# Historically moves to next page; now usually acts like a new line
text = "Page1\fPage2"
print(text)
# Output (typical console):
# Page1
# Page2
# Explanation: \f is a form-feed character, often rendered as a line break

# 8. \ooo → Octal Value
# Represents a character using octal ASCII value
text = "\110\145\154\154\157"
print(text)
# Output: Hello
# Explanation:
# \110 = H, \145 = e, \154 = l, \154 = l, \157 = o (octal ASCII values)

# 9. \xhh → Hexadecimal Value
# Represents a character using hexadecimal ASCII value
text = "\x48\x65\x6C\x6C\x6F"
print(text)
# Output: Hello
# Explanation:
# \x48 = H, \x65 = e, \x6C = l, \x6C = l, \x6F = o (hex ASCII values)
```



Thank you